

Fiori Elements 開發課程 1：環境介紹 + 連線設定

Lecture

這一課要解決什麼問題

RAP Cloud 課程 rc08 已經幫 `ZI_RC01_TASK` / `ZI_RC05_NOTE` 寫好完整的 `@UI.*` Annotation，Service Binding 也已經 Publish，Eclipse 裡點一下「Preview for Fiori Elements App」就能看到成品——但那只是一個臨時預覽：畫面是 Eclipse 內嵌瀏覽器開出來的一個 `iframe`，關掉就沒了，不會在你的電腦上留下任何檔案，也沒辦法打開程式碼看 `manifest.json` 長什麼樣、沒辦法加一個自訂按鈕、沒辦法把它變成一個能部署到正式環境的 App。

這門課要做的第一件事，就是把「Annotation 已經寫好、Service 已經 Publish」這個半成品，變成一個真正躺在你電腦硬碟上的 **App 專案**——用 VS Code + SAP Fiori tools 產生一個完整的資料夾（`manifest.json` / `webapp/` / `ui5.yaml`.....），本機執行、瀏覽器打開看到跟 Eclipse Preview 一樣（甚至更完整）的畫面，而且這個專案是你可以繼續開發、繼續客製化、最終部署出去的東西。

	Eclipse ADT「Preview for Fiori Elements App」（rc08 用過的）	這一課：VS Code + SAP Fiori tools
產生什麼	什麼都不產生，純瀏覽器 <code>iframe</code> 預覽	一個完整的本機專案資料夾（原始碼）
能不能修改畫面	不能，只能看	能，改 <code>webapp/</code> 底下的檔案就能客製化
能不能部署	不能	能（ <code>fiori deploy</code> ，這門課 fe06 教）
需要什麼工具	Eclipse ADT（RAP Cloud 課程已經在用）	VS Code + 一個免費 Extension Pack（不需要額外訂閱 SAP Business Application Studio）
連線對象	跟 Eclipse 用同一個 ADT 連線	直接連 Service Binding 曝露出來的 OData 端點（跟 Eclipse 是完全獨立的另一條連線路徑，本課重點）

安裝：VS Code Marketplace 搜尋「SAP Fiori tools - Extension Pack」

在 VS Code 左側 Extensions 面板搜尋 **SAP Fiori tools - Extension Pack**，點 Install。這是官方打包好的一組擴充套件（包含 Application Generator、Application Modeler、Service Modeler、Annotation Modeler、Guided Development、Environment Check 等），裝這一個就夠，不用逐一挑。

安裝完成後可以用 **Fiori: Environment Check**（Command Palette 打開，`Ctrl+Shift+P`）確認必要的 Node.js / npm 版本符合需求。

為什麼不需要 SAP Business Application Studio（BAS）

官方文件與教材常常預設你在用 BAS (雲端瀏覽器版的開發環境)，但 BAS 是要另外在 BTP 訂閱、啟用的服務。這門課完全不需要——SAP Fiori tools 的核心元件 (Application Generator、Proxy Middleware) 是獨立的 npm 套件 / VS Code 擴充套件，裝在自己電腦的 VS Code 上就能用，跟 Eclipse ADT 一樣是「本機工具連遠端系統」的模式，不需要額外的雲端開發環境。

Fiori Generator 精靈完整操作 Step by Step

這是這一課的核心，逐步操作 + 每一步「為什麼要這樣選」的原因 (其中三步是實際踩過坑才確認的，[[fiori-elements-tooling]] 已完整記錄)：

1. **Command Palette** (**Ctrl+Shift+P**) → **Fiori: Open Application Generator**
2. **Template** 選 **List Report Page**——這是 Fiori Elements 最常見、最標準的範本 (清單 + 明細兩頁)，這門課後面幾課 (fe03 List Report / Object Page 深化、fe08 期末 Composition App) 都是基於這個範本延伸，先從最基本的開始。
3. **Data Source** 選 **Connect to a System**：
 - ⚠ 不要選 **Connect to an OData Service**——那個選項要求直接輸入 URL + Basic Auth 帳號密碼。這個 BTP Trial 的個人開發者帳號完全不支援 **Basic Auth** (RAP Cloud 課程 rc07 用 Postman 測過同樣的限制：直接 Basic Auth 打 API 只會被導向 OAuth 登入頁)，選了這個選項精靈會卡在認證這一步過不去。
 - **Connect to a System** 才是給「需要走瀏覽器登入」的雲端系統用的選項。
4. **System** 選 **New System**，**System Type** 選 **ABAP Environment on SAP Business Technology Platform**。

⚠ 如果你之前已經用同一個 **URL / Client** 建立過連線 (例如做過本課環境驗證時建立的 **TRL**)，這裡選 **New System** 重新輸入同一組 **URL** 會報錯：A saved system connection entry: "TRL" already exists with the same URL and client. Please reuse the existing entry or remove it. ——不需要刪除重建，直接把 **System** 欄位改選既有的 **TRL** 那筆 (下拉選單裡找得到) 即可，效果完全一樣，還能跳過後面重新輸入 System URL / 重新登入的步驟。真的想砍掉重建才需要去 VS Code 左側 Activity Bar 的鎖頭圖示 (**Connection Manager for SAP Systems**)，點開會列出所有已儲存的連線 (例如 **TRL (ABAP Cloud)** [CB9980001705])，滑鼠移到該筆連線上會出現垃圾桶圖示 (**Delete SAP System**)，點掉即可刪除。

1. (只有選 **New System** 才需要這幾步；選既有系統會直接跳過，從第 8 步接續) **ABAP environment definition source** 選 **Use Reentrance Ticket**：
 - ⚠ 不要選 **Discover a Cloud Foundry Service**——那個需要你先在終端機 cf login 登入 Cloud Foundry，還要有這個 BTP Trial 帳號的 CF Space 存取權限，設定成本高很多，這個共用帳號也沒有走過這條路。
 - Reentrance Ticket 是 SAP 雲端系統給「已經用瀏覽器登入過」的使用者簽發的一次性票證，機制上等同「幫我用目前瀏覽器已登入的身分授權這個工具」，不需要另外管理帳密或 CF 權限。
2. ⚠ ⚠ **System URL** 是全流程最容易填錯的地方：
 - 不能填瀏覽器平常開 Swagger UI / ADT 用的網域 (<https://<GUID>.abap-web.ap21.hana.ondemand.com>) ——這個 **abap-web.*** 網域是給瀏覽器互動用的，Fiori Generator 拿它去要 OData \$metadata 會直接失敗，報錯「Edmx metadata is required to continue with generation」(曾經以為是官方未解的已知 bug，查過 SAP Community 才確認純粹是網址填錯)。
 - 要填 BTP Service Key JSON 裡 "url" 欄位那個網域：<https://<同一組GUID>.abap.ap21.hana.ondemand.com> (注意沒有 **-web**) ——這才是真正給程式化存取用的 API 網域。

- Service Key 在 BTP Cockpit 的 Instances and Subscriptions → 找到 ABAP Environment 的 Instance → Service Keys 分頁可以查到，或問系統管理者要這組 URL。
3. **System name** 隨意命名（例如 **TRL**），按 Next 會跳出瀏覽器登入視窗，用平常登入這個 Trial 帳號的方式登入即可。
 4. **Service** 欄位（畫面上實際標示是 **Service (for user [使用者技術帳號，如 CB9980001705>])**）是一個文字搜尋框（不是選了既有 **System** 就變成瀏覽式下拉選單），要打對的關鍵字才會搜到：☒ 已實測確認，輸入的是 **Service Binding** 的技術名稱，不是 **Service Definition**——這一課要打 **ZRC08_SB**（不是 **ZRC08_SD**），打對才搜得到、跳出正確的服務。
 - 判斷依據（實測前的推論）：**manifest.json** 產生後 **sap.app.dataSources.mainService.uri** 長這樣：**/sap/opu/odata4/sap/zrc08_sb/srvd/sap/zrc08_sd/0001/**——路徑第一段（緊接在 **/sap/opu/odata4/sap/** 後面）是 **Service Binding (zrc08_sb)**，OData 服務目錄 (Catalog) 真正拿來註冊、對外曝露的就是這個名字；Service Definition (**zrc08_sd**) 是它裡面的下一層，不是搜尋的入口。
 - 搜尋結果會顯示成 **ZRC08_SB > ZRC08_SD (0001)** 這種「Binding > Definition (版本)」的格式，看到這筆就對了——這一課選 **ZRC08_SB > ZRC08_SD (0001)**（rc08 建立、曝露 **ZI_RC05_NOTE** 的那組，Draft-enabled，Preview 有完整 Create / Edit 按鈕）。
 - **打 ZRC08_SD (Service Definition 名稱) 搜不到**——這是本課實測驗證過的具體結論，不是單純理論推導，之後任何一課要連新的 Service，都要記得用 Service Binding 名稱去搜。
 5. 「**Download value help metadata**」選 **Yes**——實際畫面問句是：「The service contains references to value help services. Do you want to download the associated metadata during generation? This may increase generation time.」，意思是精靈偵測到這個 Service 的欄位上掛了 Value Help (**ZI_RC05_NOTE** 有沒有掛 Value Help 這一課不一定用得到，但選 **Yes** 保險起見一次連同下載，只是生成時間會稍微變長；選 **No** 之後如果真的用到 Value Help 欄位，畫面可能無法正常帶出選項，所以固定選 **Yes**)。
 6. **Entity Selection**：Main Entity 選 **Note**；Table Type 保留預設 **Responsive**（資料量小、不需要階層或聚合顯示，**Responsive** 是最通用的表格型態）。
 7. **Project Attributes**：
 - Module Name 填 **fe01_connection_test**（這門課的物件命名慣例：每一課一個獨立前端專案，**feNN_<主題>**）
 - Application Title 自訂（例如「FE01 連線測試」）
 - TypeScript / Deployment Config / Launchpad Config / Advanced Options 全部保留預設 **No**
 - **Use Virtual Endpoints for Local Preview** 保留預設 **Yes**
 - **Project Folder Path**：**這一課要指定成 **src/ABAP_Training_Fiori_Elements/**（這個 repo 底下），不要放到 repo 外面的隨意資料夾——這樣專案才會被版控留存，符合這門課「每一課一個獨立專案資料夾」的教材慣例**
 - 按 **Finish**

產生完成後：這才是「真正的專案」

精靈跑完，**src/ABAP_Training_Fiori_Elements/fe01_connection_test/** 底下會出現一個完整的資料夾，包含（細節這門課 fe02 會逐項拆解）：

- **webapp/manifest.json**——App 的核心設定檔，**sap.app.dataSources.mainService.uri** 會指向 **/sap/opu/odata4/sap/zrc08_sb/srvd/sap/zrc08_sd/0001/** 這個真實的 OData V4 端點
- **webapp/annotations/annotation.xml**——精靈下載回來的 **@UI.*** Annotation（跟你在 rc08 寫在 ABAP 端的 Metadata Extension 是同一份內容，只是轉成 OData 標準的 Annotation XML 格式）

- `ui5.yaml`——本機開發伺服器設定，最關鍵的是 `fiori-tools-proxy middleware`，裡面的 `backend.url` 就是第 6 步填的那個 API 網域，`authenticationType: reentranceTicket` 代表本機執行時一樣走 Reentrance Ticket 認證
- `package.json`——npm 專案設定，`scripts.start` 對應到 `fiori run --open "test/flp.html#app-preview"`

這個資料夾本身就是完整的原始碼——跟 Eclipse Preview 那個「關掉就沒了」的 `iframe` 完全不同，這裡的每一個檔案都可以用 VS Code (或 Claude) 打開來讀、改、版控。

用 **Application Info** 面板快速核對專案設定 (不用逐檔翻 `manifest.json`)

SAP Fiori tools 內建一個 **Application Info** 面板 (Command Palette 打 **Fiori: Open Application Info**，或點任一檔案 Editor 右上角的橘色火焰圖示)，一次彙整顯示這個專案最關鍵的設定：`Identifier / Title / Namespace / Min UI5 Version / Main Service` (例如這一課會顯示 `zrc08_sd (V4.0)`)、`Files` 底下還有 `package.json / manifest.json` 的捷徑連結，點了直接跳到對應檔案。

⚠ 這個面板本身不是檔案，是 **VS Code 擴充套件動態渲染的 Webview**——畫面上每一項都是即時解析 `webapp/manifest.json / package.json` 等既有檔案得到的結果，不會另外產生任何 `.html` 存進專案資料夾，**不需要 (也不會) 進版控**。它的價值純粹是「診斷用的儀表板」：連線設定改完之後，與其自己打開 `manifest.json` 找 `dataSources.mainService.uri` 一行行核對，不如直接開這個面板一眼確認 **Main Service** 對不對、UI5 版本對不對——特別適合這一課驗證「Service 有沒有搜對 / 連對」的場景。

本機執行：`npm start` 跟 Fiori Tools Proxy 怎麼運作

1. 在專案資料夾下執行 `npm install` (第一次要抓依賴套件)
2. 執行 `npm start` (等同 `package.json` 裡的 `fiori run --open "test/flp.html#app-preview"`)
3. VS Code 會在本機啟動一個開發伺服器 (預設 `http://localhost:8080`)，瀏覽器自動開啟 `http://localhost:8080/test/flp.html#app-preview`

這中間發生的事：瀏覽器打的其實是本機的 `localhost:8080`，但畫面上看到的是真實的 **BTP 後端資料**——原因是 `ui5.yaml` 裡的 `fiori-tools-proxy middleware` 攔截了所有 `/sap/...` 開頭的請求，用 Reentrance Ticket 認證，即時轉發到真正的 `https://<GUID>.abap.ap21.hana.ondemand.com`。這跟 Eclipse Preview 「直接連正式環境看資料」的效果一樣，但現在是**你自己的本機專案在跑**，不是 Eclipse 借用的臨時畫面。

Proxy 存在的原因、用途、實際運作方式 (實測 log 逐行對照)

為什麼需要 **Proxy**，不能讓瀏覽器直接打真實後端？

1. **跨網域 (CORS)**：瀏覽器頁面是 `http://localhost:8080` 載入的，OData 服務在完全不同的網域 `https://<GUID>.abap.ap21.hana.ondemand.com`——瀏覽器的同源政策 (Same-Origin Policy) 預設會擋掉這種跨網域的 XHR/Fetch 請求，除非後端明確開放 CORS。與其要求後端配合開放，業界標準做法是本機起一個「代理伺服器」，讓瀏覽器只跟 `localhost:8080` 溝通 (同源，不受限)，由這個本機伺服器 (Node.js 執行的 Proxy) 代替瀏覽器去跟真正的後端要資料，再把結果轉交回瀏覽器。
2. **認證流程需要一個能保存 Session / Cookie 的地方**：Reentrance Ticket 走的是瀏覽器互動式登入 (OAuth 導向)，登入成功後後端會核發一組 Session Cookie / Token。如果每次都靠瀏覽器裸打跨網域請求，瀏覽器的 Cookie 隔離機制會讓這組認證資訊很難乾淨地在 `localhost` 跟遠端網域之間共用。Proxy 這一層可以把「登入這件事」統一在 Node.js process 裡處理一次、快取起來，之後的請求都用同一份憑證轉發，瀏覽器端完全不用管認證細節。

這一課實測 `npm start` 的 `log`，逐段對照 `ui5.yaml` 的設定，可以看到 **Proxy** 具體做了什麼：

```
info fiori-tools-proxy backend:
[{"path":"/sap","url":"https://a396c05d-....abap.ap21.hana.ondemand.com","authenticationType":"reentranceTicket"}]
info fiori-tools-proxy ui5: [{"path":"/resources","url":"https://ui5.sap.com", ...}, {"path":"/test-resources","url":"https://ui5.sap.com", ...}]
```

`ui5.yaml` 其實設定了兩條完全獨立的轉發規則，不是只有連後端資料這一條：

- **backend** 規則 (`path: /sap` → 你的真實 BTP 後端)：轉發 OData 業務資料的請求 (`$metadata`、實際的 CRUD)。
- **ui5** 規則 (`path: /resources /test-resources` → `https://ui5.sap.com`)：這條容易被忽略但同樣重要——瀏覽器載入的 UI5 Framework 本身 (`sap.m / sap.fe.templates` 這些函式庫檔案) 也是透過 Proxy、即時從 SAP 官方 CDN 抓的，不是從本機 `node_modules` 載入。這樣可以確保本機開發時用的 UI5 版本，跟 `manifest.json` 裡 `minUI5Version` 宣告的版本一致，不用自己在專案裡管理一份 UI5 執行期函式庫。

```
warn backend-proxy-middleware No credential found. Service: [fiori/v2/system], Key:
[https://a396c05d-....abap.ap21.hana.ondemand.com]
info backend-proxy-middleware Backend proxy created for
https://a396c05d-....abap.ap21.hana.ondemand.com
```

`No credential found` 不是錯誤——這是 Proxy 剛啟動、還沒有任何請求真正打過去時的正常訊息 (此時連 Reentrance Ticket 登入流程都還沒觸發，因為瀏覽器連第一個 OData 請求都還沒送出)。真正的登入是**延遲觸發**的：等瀏覽器實際發出第一個需要認證的請求，Proxy 才會啟動 Reentrance Ticket 的登入流程 (跳出瀏覽器登入視窗)，成功後把拿到的憑證快取在這個 `npm start` 執行中的 Node.js process 記憶體裡，之後同一個 process 存活期間的請求都直接複用，不會每個請求都重新登入一次。

```
info backend-proxy-middleware Rewrite path
/opu/odata4/sap/zrc08_sb/srzd/sap/zrc08_sd/0001/ >
/sap/opu/odata4/sap/zrc08_sb/srzd/sap/zrc08_sd/0001/
```

這一行是「**Virtual Endpoint**」機制的具體證據：Generator 精靈第 11 步保留預設 **Yes** 的 **Use Virtual Endpoints for Local Preview**，讓本機預覽時 App 實際發出的請求路徑是不帶 `/sap` 前綴的 `/opu/odata4/sap/zrc08_sb/...`；Proxy 收到之後才把 `/sap` 補回去，轉發到後端真正認得的路徑 `/sap/opu/odata4/sap/zrc08_sb/...`。這一層轉換讓本機開發時的 URL 更乾淨 (不用在每個相對路徑前面重複打 `/sap`)，實際打去後端的路徑才是完整、正確的。

整體資料流總結：

```
瀏覽器 (http://localhost:8080)
| 同源請求，不受 CORS 限制
▼
本機 fiori-tools-proxy (npm start 起的 Node.js process)
| ① UI5 Framework 檔案 → 轉發到 https://ui5.sap.com
```

| ② OData 業務資料 → 補回 /sap 前綴、帶 Reentrance Ticket 憑證 → 轉發到
https://<GUID>.abap.ap21.hana.ondemand.com

▼
真實 BTP ABAP Environment 後端 (跟 Eclipse ADT 連的是同一個系統，只是完全獨立的另一條連線路徑)

⚠ **Reentrance Ticket 有快取但可能過期**：第一次執行通常會跳出瀏覽器登入視窗；如果同一台電腦不久前才登入過 (VS Code Fiori tools 本機有快取登入狀態)，可能不會再跳出登入視窗，直接顯示資料。如果隔了一段時間、Token 過期，重新執行 `npm start` 應該會再跳出登入視窗，正常重新登入即可。這個快取是跟著 `npm start` 這個 process 走的——關掉終端機、重新執行 `npm start`，等於是一個全新的 Node.js process，之前快取的憑證不會保留，大機率會重新觸發登入流程。

學習目標

- 能講出「Eclipse ADT Preview」跟「VS Code Fiori tools 產生的真實專案」本質上的差異 (一個是臨時 iframe，一個是可持續開發/部署的本機原始碼)
- 能獨立安裝 SAP Fiori tools Extension Pack，知道不需要額外訂閱 SAP Business Application Studio
- 能跑完整個 Fiori Application Generator 精靈，並講出三個關鍵坑分別是什麼、為什麼會卡住：
 - Data Source 一定要選 **Connect to a System** (不能選 **Connect to an OData Service**，因為 Trial 帳號不支援 Basic Auth)
 - Authentication 要選 **Use Reentrance Ticket** (不是 **Discover a Cloud Foundry Service**)
 - System URL 要用 Service Key 裡的 **abap.*** API 網域 (不是瀏覽器用的 **abap-web.*** 網域)
- 知道生成的專案資料夾裡 `manifest.json` / `ui5.yaml` 各自的角色 (下一課 fe02 會逐項細看)
- 能執行 `npm start`，理解 `fiori-tools-proxy` middleware 如何讓本機開發伺服器串接真實後端資料

物件清單

這一課不建立任何新的 ABAP 後端物件，完全沿用 RAP Cloud 課程已經建好並 Publish 的物件：

物件	名稱	型別	來源
CDS Root View Entity (Draft)	ZI_RC05_NOTE	DDLS/DF	rc05
Service Definition	ZRC08_SD	SRVD/SRV	rc08
Service Binding (OData V4 - UI，已 Publish)	ZRC08_SB	SRVB/SVB	rc08

前端新產生的物件：

物件	說明
fe01_connection_test/	這一課產生的 Fiori Elements App 專案，存於 src/ABAP_Training_Fiori_Elements/fe01_connection_test/

動手練習

輪到你了——這一課的「動手練習」就是實際操作本身，因為 Fiori Application Generator 是互動式精靈 (Yeoman-based)，沒有對應的自動化工具，這一步一定要你在 VS Code 裡親自跑一次：

1. 安裝 SAP Fiori tools - Extension Pack (如果還沒裝)
2. 照上面「Fiori Generator 精靈完整操作 Step by Step」一步步做，Project Folder Path 指到 `src/ABAP_Training_Fiori_Elements/`，Module Name 填 `fe01_connection_test`
3. 完成後跟我說一聲，我會讀取產生的檔案 (`manifest.json` / `ui5.yaml` / `package.json`) 核對內容是否符合預期
4. 接著在專案資料夾下執行 `npm install + npm start`，瀏覽器截圖回報看到的畫面

加碼練習 (選做)：重複一次同樣流程，這次 Service 選 `ZRC07_SB > ZRC07_SD (0001)`、Main Entity 選 Task (rc01 的非 Draft 實體)，比較這個新專案跟 `fe01_connection_test` 的 List Report 畫面差在哪裡 (提示：rc08 已經證實過 Draft / 非 Draft 在 Create / Edit 按鈕上的差異，這次換成在自己本機的真實專案裡重新驗證一次同樣的結論)。

驗證方式

這一課沒有 ABAP 語法檢查 / 啟用可以驗證 (不涉及後端物件變更)，驗證重點是專案結構正確 + 本機真的連得到真實資料：

1. Claude 核對 `webapp/manifest.json` 的 `sap.app.dataSources.mainService.uri` 正確指向 `/sap/opu/odata4/sap/zrc08_sb/srvd/sap/zrc08_sd/0001/`
2. Claude 核對 `ui5.yaml` 的 `fiori-tools-proxy` middleware 設定正確 (`backend.url` 是 `abap.*` 網域、`authenticationType: reentranceTicket`)
3. 使用者執行 `npm start`，瀏覽器截圖確認：
 - `http://localhost:8080/test/flp.html#app-preview` 正常開啟，沒有卡在登入或報錯畫面
 - List Report 顯示真實的 Note 資料 (不是空白或錯誤訊息)
 - 工具列有 **Create** 按鈕、篩選列有 **Editing Status** (Draft 專屬，rc08 已經驗證過的特徵，這次在本機專案裡重新確認一次)
4. (選用，比翻檔案更快) 開 Application Info 面板 (**Fiori: Open Application Info**)，一眼核對 Main Service 顯示 `zrc08_sd (V4.0)`、Min UI5 Version 跟 `manifest.json` 一致——效果跟第 1 點手動核對 `manifest.json` 一樣，只是不用自己找檔案

這一課實測結果：`fe01_connection_test` 的 `manifest.json` / `ui5.yaml` 內容核對正確，`npm start` 本機執行成功，瀏覽器 List Report 正常顯示真實資料，Create 按鈕與 Editing Status 篩選都有出現，Application Info 面板也確認 Main Service 正確對到 `zrc08_sd (V4.0)`。

思考題

1. `ui5.yaml` 裡 `fiori-tools-proxy` 的 `backend.url` 填的是 Service Key 裡的 `abap.*` 網域——如果你把這個 URL 換成 `abap-web.*` 網域重新執行 `npm start`，你覺得會發生什麼事？ (提示：想一想這兩個網域在 Generator 精靈那一步分別扮演什麼角色，Proxy 的行為跟 Generator 產生 metadata 這一步是不是同一件事)
2. 這一課完全沒有碰任何 ABAP 物件，但畫面上看到的 Create / Editing Status 這些行為，本質上是誰決定的——是這個 VS Code 專案的程式碼決定的，還是後端 `ZI_RC05_NOTE` 的 `with draft` 決定的？如果答案是後端，那 VS Code 這邊產生的 `manifest.json` / `annotation.xml` 扮演的角色又是什麼？
3. 這一課登入用的是 Reentrance Ticket，跟你自己的個人帳號綁定；如果同事想要開發同一個 App，他能不能直接拿走你這個 `fe01_connection_test` 資料夾就開始用？中間需要換掉/補上什麼東西？ (提示：想想 Reentrance Ticket 的性質，跟你的登入身分有沒有關係)

答案

見 `fe01_connection_test/webapp/manifest.json`、`fe01_connection_test/ui5.yaml`、`fe01_connection_test/package.json` (前端專案的關鍵設定檔快照；`node_modules/ / dist/` 等產生物依專案自帶的 `.gitignore` 排除，不進版控)。後端沿用 rc08 已 Publish 的 `ZRC08_SB` (`ZI_RC05_NOTE`)，沒有新增或修改任何 ABAP 物件。實測結果：`npm start` 本機執行，`http://localhost:8080/test/flp.html#app-preview` 正常顯示真實資料，Create 按鈕與 Editing Status 篩選都正常出現。